



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Varrel Fernando
NPM	2428240068
Kelas	SI5B
Topik Tugas 1	museum
Entitas utama	Koleksi museum
Tautan repositori	https://github.com/Lyvarion-18/PAW2_si5b_kuis1_varrelFernando
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	08-10-2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Museum
Entitas utama (tunggal)	Koleksi Museum
Alamat dasar	/museum-collections
Field data	namaKoleksi (string), kategori (string), periode (string), asalDaerah (string), tahunDitemukan (number)
Field wajib saat POST	namaKoleksi, kategori, periode
Query string untuk filter	periode
Data awal	3 data koleksi museum
Modul lain (jika ada)	-

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

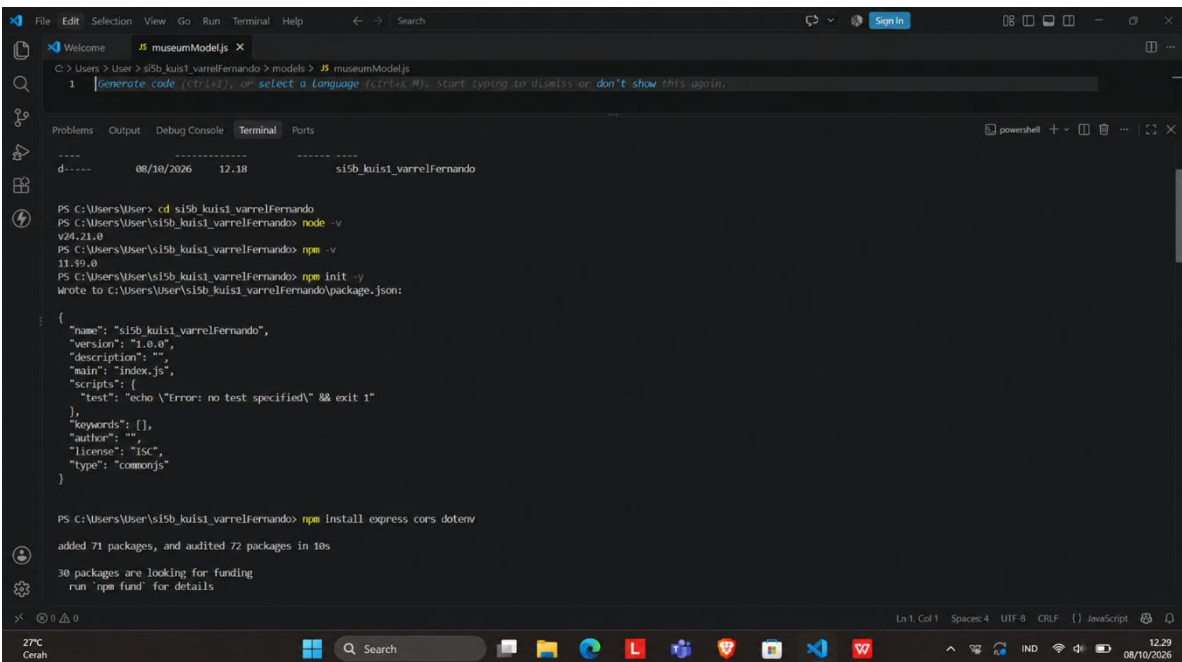
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



```
C:\Users\User> cd si5b_kuis1_varrelFernando > models > JS museumModels.js
1 | Generate code (Ctrl+I), or select a language (Ctrl+R). Start typing to dismiss or don't show this again.

-----
d----- 08/10/2026 12:18 si5b_kuis1_varrelFernando

PS C:\Users\User> cd si5b_kuis1_varrelFernando
PS C:\Users\User\si5b_kuis1_varrelFernando> node -v
v24.21.0
PS C:\Users\User\si5b_kuis1_varrelFernando> npm -v
11.99.0
PS C:\Users\User\si5b_kuis1_varrelFernando> npm init -y
Wrote to C:\Users\User\si5b_kuis1_varrelFernando\package.json:

{
  "name": "si5b_kuis1_varrelFernando",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\User\si5b_kuis1_varrelFernando> npm install express cors dotenv
added 71 packages, and audited 72 packages in 10s
38 packages are looking for funding
run 'npm fund' for details
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

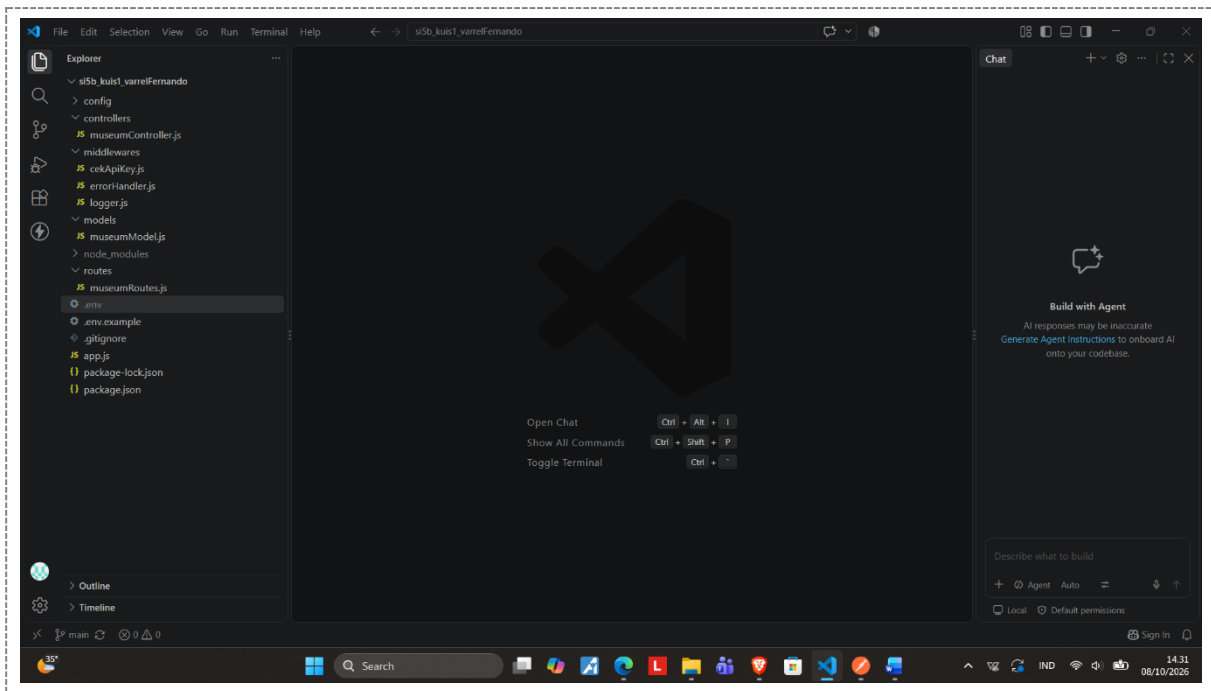
Pada tahap ini saya melakukan inisialisasi project Node.js dengan mengecek versi Node.js dan npm, kemudian membuat file package.json menggunakan `npm init -y`. Selanjutnya saya menginstal Express untuk membuat RESTful API, CORS untuk mengatur akses request dari sumber yang berbeda, dan dotenv untuk membaca konfigurasi dari file `.env`. Node.js yang digunakan adalah versi 24.21.0 dan memenuhi syarat untuk menggunakan Express 5..

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Folder routes digunakan untuk mengatur alamat endpoint API dan menghubungkannya dengan fungsi Controller. Folder controllers digunakan untuk menangani request, melakukan validasi data, dan memberikan response kepada client. Folder models digunakan untuk menyimpan data koleksi museum serta fungsi untuk mengambil, menambah, mengubah, dan menghapus data. Folder middlewares digunakan untuk menjalankan proses tambahan sebelum request diteruskan, seperti logger, pengecekan API key, dan penanganan error.

2.2 Ringkasan tugas setiap berkas

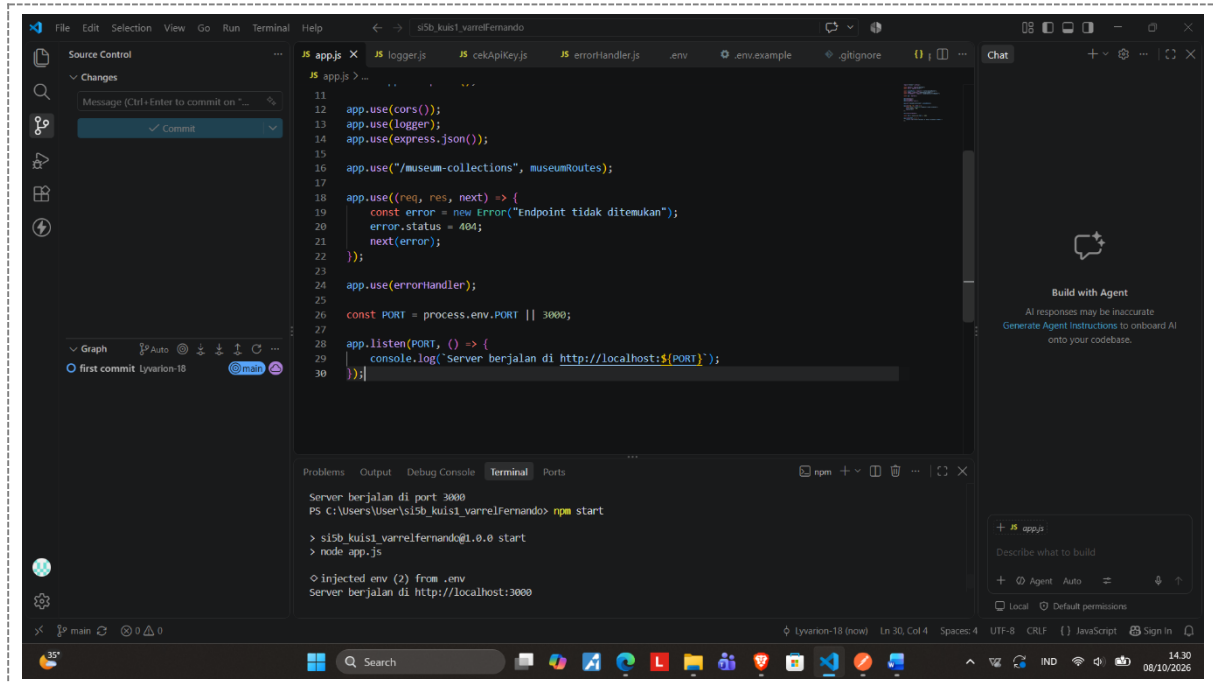
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/[sumber-daya]Model.js	File ini menyimpan data koleksi museum dalam variabel collections dan menyediakan fungsi getAllCollections, getCollectionById, addCollection, updateCollection, dan deleteCollection untuk mengolah data.
controllers/[sumber-daya]Controller.js	File ini menangani request dan response API melalui fungsi getAllCollections, getCollectionById, createCollection, updateCollection, dan deleteCollection, termasuk melakukan validasi data.
routes/[sumber-daya]Routes.js	File ini menggunakan express.Router() untuk menghubungkan endpoint /m dengan fungsi pada museumController serta menerapkan cekApiKey pada DELETE.
middlewares/logger.js	File ini digunakan untuk mencatat method dan alamat request yang masuk sebelum request diteruskan ke proses berikutnya.
middlewares/cekApiKey.js	File ini memeriksa x-api-key pada request dan membandingkannya dengan API_KEY dari konfigurasi .env sebelum request dilanjutkan.
middlewares/errorHandler.js	File ini menangani error secara terpusat dan mengubah error menjadi response JSON dengan status HTTP yang sesuai.
app.js	File ini menjadi titik utama aplikasi yang mengatur Express, CORS, express.json(), middleware, route museumRoutes, dan menjalankan server pada port yang ditentukan.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Pada tahap ini saya menjalankan server menggunakan perintah `npm start`. Server berhasil berjalan pada port 3000 sesuai dengan nilai `PORT` yang terdapat pada file `.env`.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

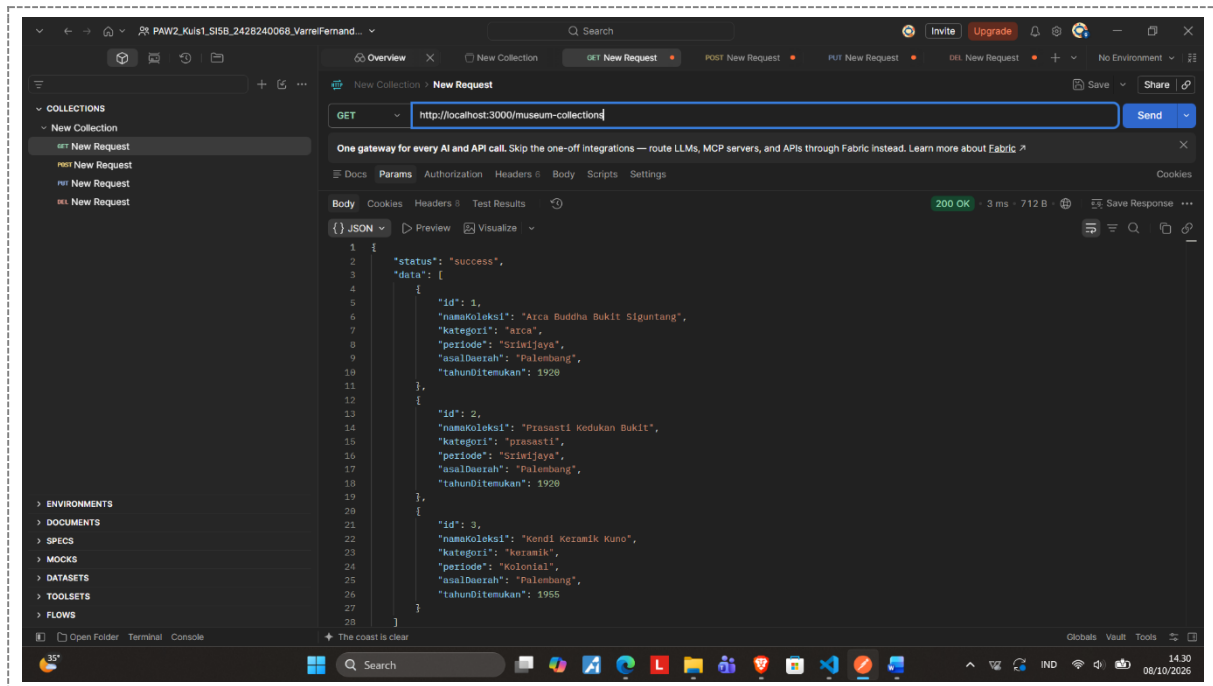
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (<i>request</i>)	GET /museum-collections
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK — Data koleksi museum berhasil ditampilkan.
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

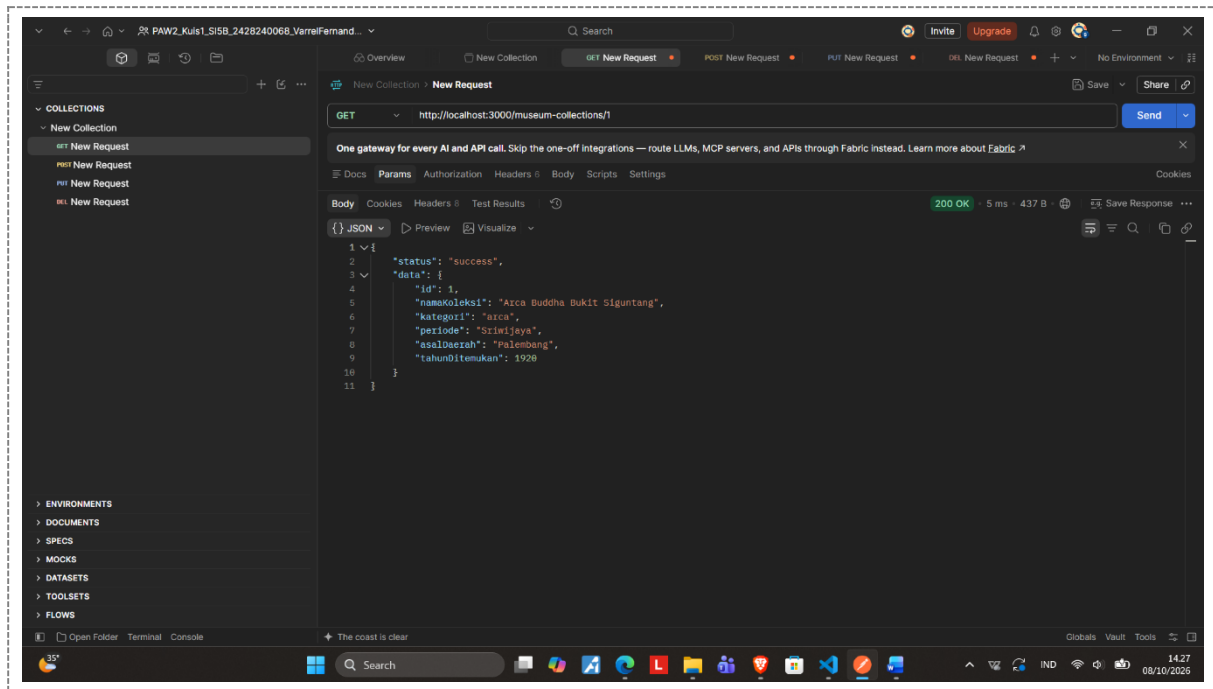
Penjelasan:

Endpoint GET digunakan untuk menampilkan seluruh data koleksi museum. Request masuk melalui museumRoutes, kemudian diteruskan ke fungsi getAllCollections pada Controller dan mengambil data melalui fungsi getAllCollections pada Model. Response berhasil dengan status 200 karena seluruh data koleksi berhasil ditampilkan.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /museum-collections/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK — Data koleksi museum dengan ID 1 berhasil ditampilkan.
Lulus?	☒ Ya ☐ Tidak



Gambar 5. Hasil GET berdasarkan ID

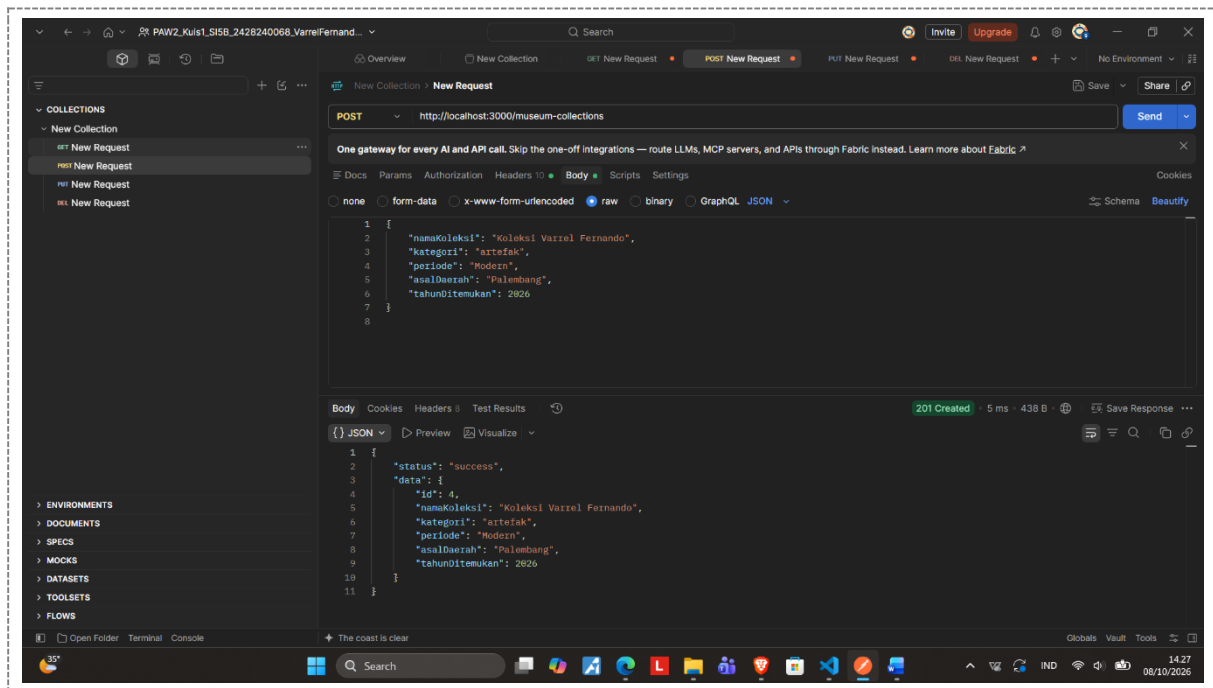
Penjelasan:

Endpoint GET berdasarkan ID digunakan untuk menampilkan satu data koleksi museum. ID diambil dari req.params.id pada Controller, kemudian diubah menjadi angka menggunakan Number() agar dapat dibandingkan dengan ID pada Model. Data dengan ID 1 berhasil ditemukan sehingga server memberikan response dengan status 200.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /museum-collections
Header	x-api-key: varrel123 Content-Type: application/json
Body	<pre>{ "namaKoleksi": "Koleksi Varrel Fernando", "kategori": "artefak", "periode": "Modern", "asalDaerah": "Palembang", "tahunDitemukan": 2026 }</pre>
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created — Data koleksi museum berhasil ditambahkan.
Lulus?	☒ Ya ☐ Tidak



Gambar 6. Hasil POST data baru

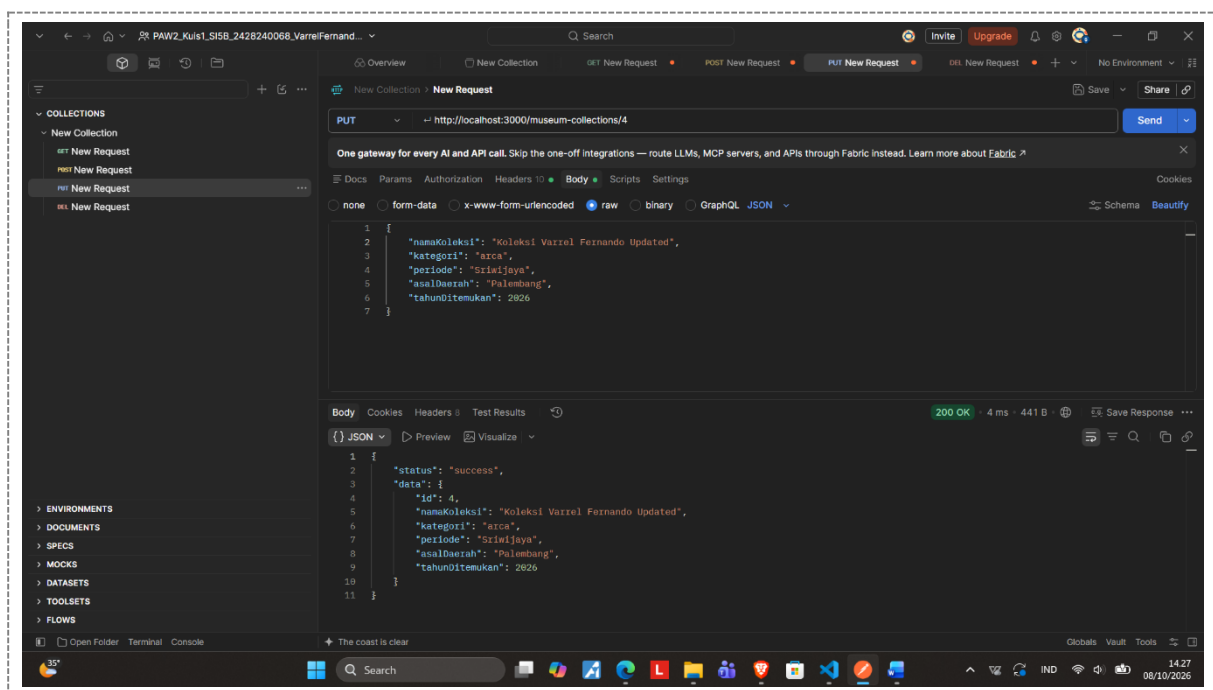
Penjelasan:

Endpoint POST digunakan untuk menambahkan data koleksi museum baru. Request diperiksa oleh middleware cekApiKey melalui header x-api-key, sedangkan express.json() digunakan untuk membaca body dalam format JSON. Data yang valid kemudian diproses oleh Controller dan Model, lalu server memberikan status 201 karena data baru berhasil dibuat.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /museum-collections/4
Header	x-api-key: varrel123 Content-Type: application/json
Body	{ "[field]": "[nilai baru]" }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK — Data koleksi dengan ID 4 berhasil diperbarui.
Lulus?	☒ Ya ☐ Tidak



Gambar 7. Hasil PUT mengubah data

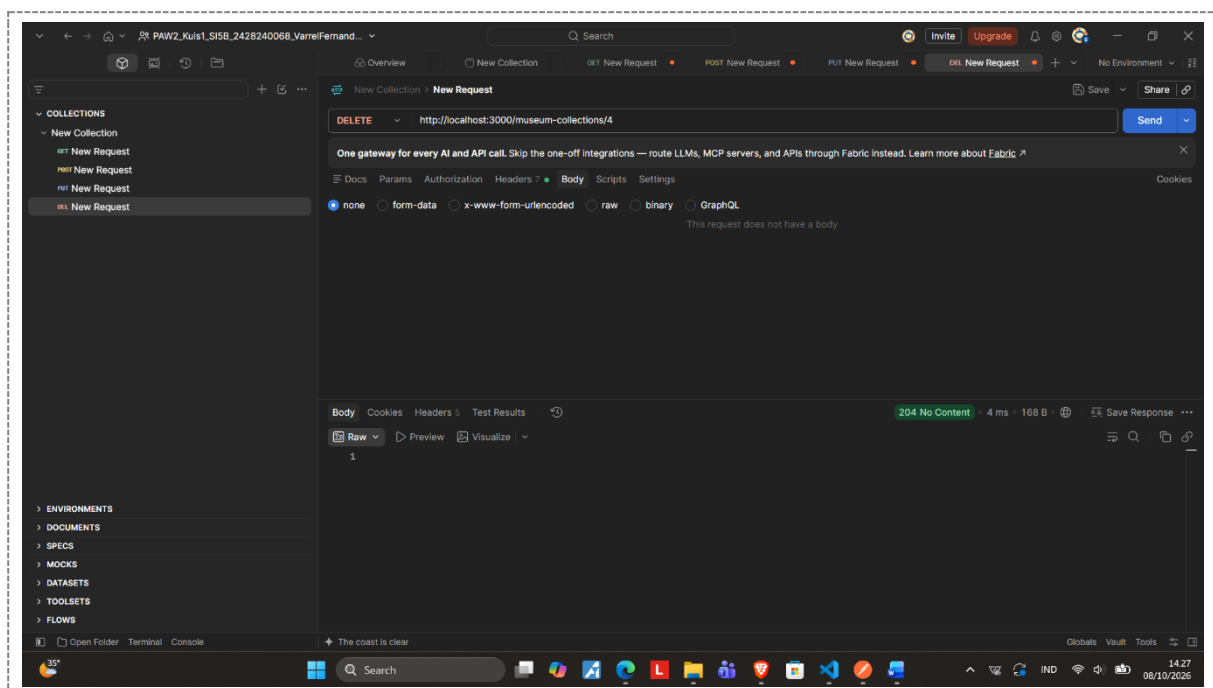
Penjelasan:

Endpoint PUT digunakan untuk mengubah data koleksi museum dengan ID 4. Controller mengambil ID dari `req.params.id` dan data baru dari `req.body`, kemudian meneruskannya ke fungsi `updateCollection` pada Model untuk mengubah data. Pada pengujian ini beberapa field data berubah dan server memberikan response dengan status 200.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /[sumber-daya]/[id data dari T03]
Header	x-api-key: (nilai API_KEY pada .env Anda)
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☒ Ya ☐ Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Endpoint DELETE digunakan untuk menghapus data koleksi museum berdasarkan ID. Request diperiksa terlebih dahulu oleh middleware cekApiKey, kemudian Controller memanggil fungsi deleteCollection pada Model untuk menghapus data dengan ID 4. Response menggunakan status 204 karena data berhasil dihapus dan server tidak mengirimkan isi response

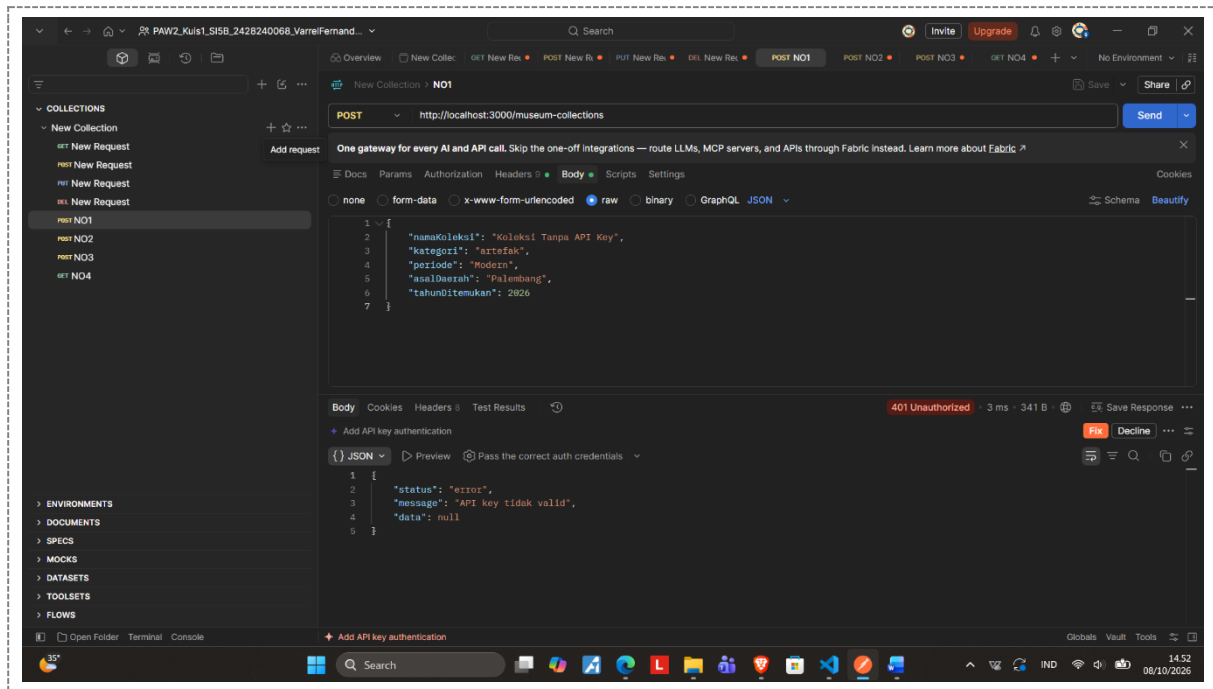
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /museum-collections
Header	Content-Type: application/json
Body	<pre>{ "namaKoleksi": "Koleksi Tanpa API Key", "kategori": "artefak", "periode": "Modern", "asalDaerah": "Palembang", "tahunDitemukan": 2026 }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized — API key tidak valid.
Lulus?	☒ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

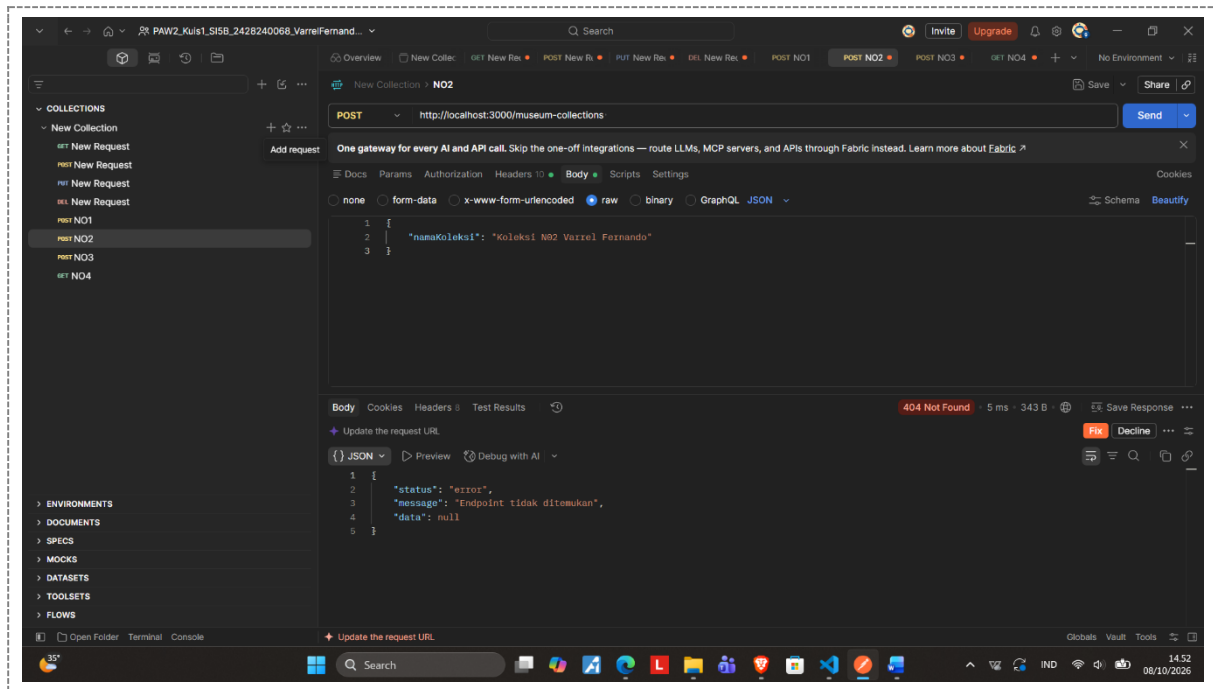
Penjelasan:

Request POST tanpa header x-api-key ditolak oleh middleware cekApiKey sebelum masuk ke Controller. Middleware memeriksa API key dan memberikan response dengan status 401 karena API key tidak ditemukan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /museum-collections
Header	x-api-key: varrel123 Content-Type: application/json
Body	<pre>{ "namaKoleksi": "Koleksi N02 Varrel Fernando"}</pre>
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request — namaKoleksi, kategori, dan periode wajib diisi.
Lulus?	☒ Ya ☐ Tidak



Gambar 10. POST dengan body tidak lengkap

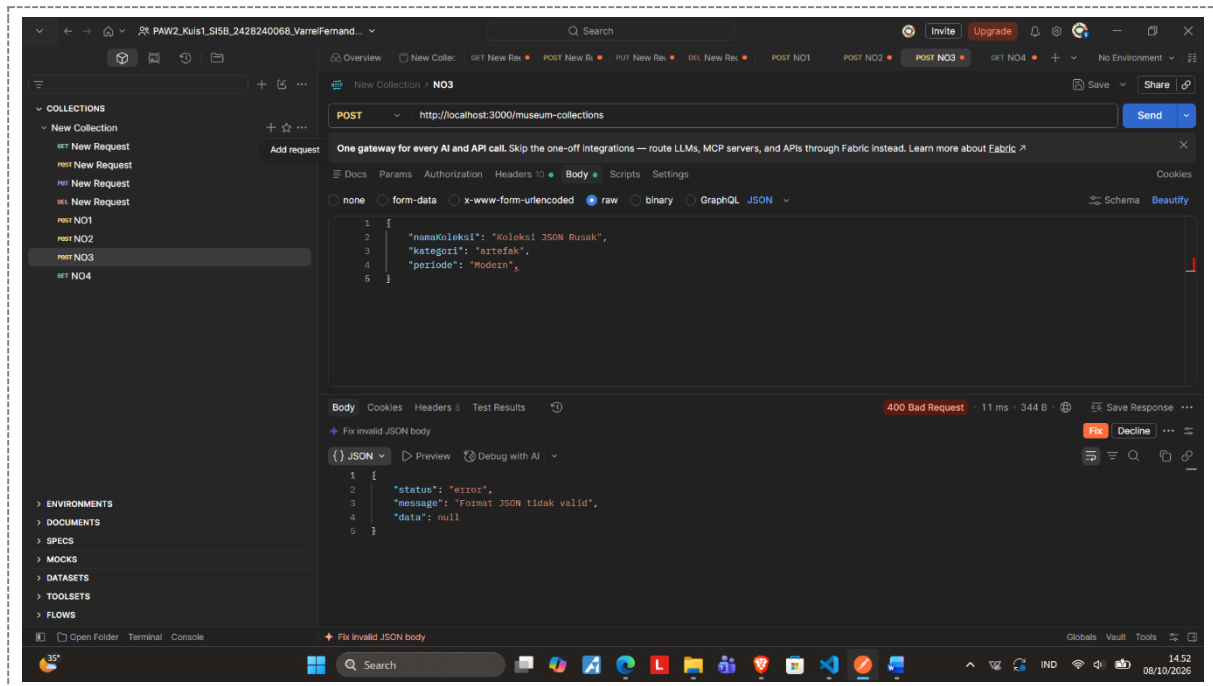
Penjelasan:

Validasi data dilakukan pada Controller sebelum data diteruskan ke Model. Karena request tidak memiliki seluruh field wajib yaitu namaKoleksi, kategori, dan periode, server memberikan response dengan status 400 Bad Request

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /museum-collections
Header	x-api-key: varrel123 Content-Type: application/json
Body	<pre>{ "namaKoleksi": "Koleksi JSON Rusak", "kategori": "artefak", "periode": "Modern", }</pre>
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request — Format JSON tidak valid.
Lulus?	☒ Ya ☐ Tidak



Gambar 11. POST dengan JSON rusak

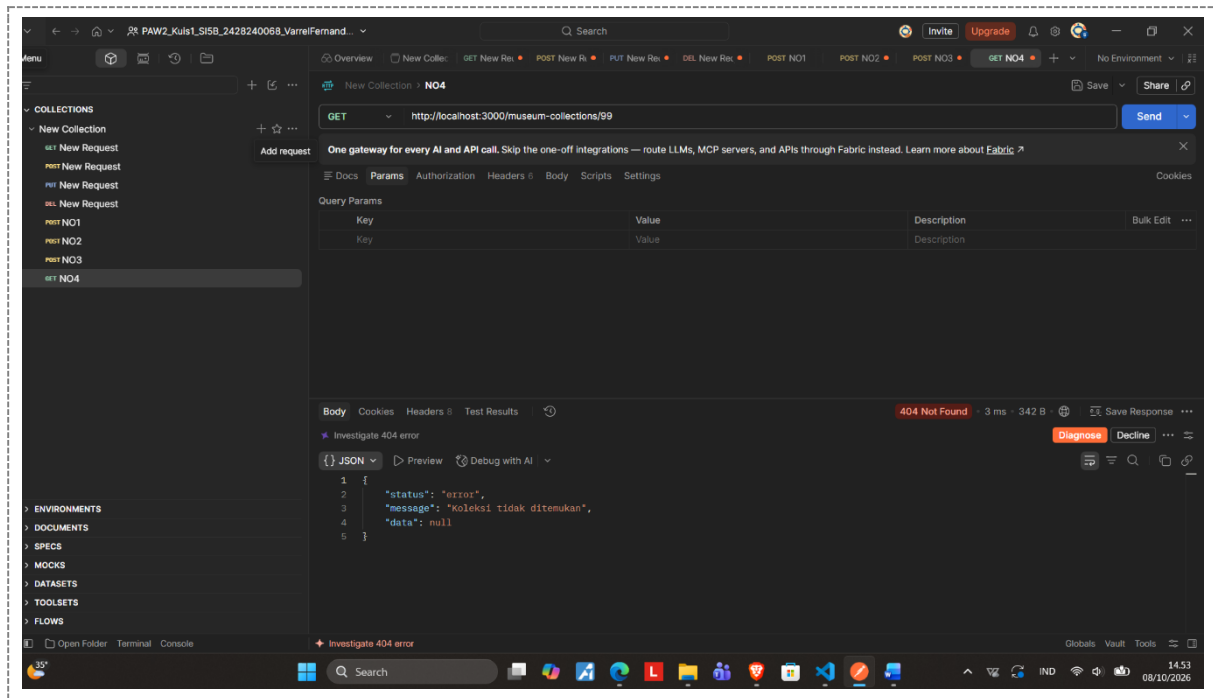
Penjelasan:

JSON yang dikirim tidak dapat diproses oleh `express.json()` karena formatnya tidak valid. Error tersebut diteruskan ke `errorHandler` yang mengubahnya menjadi response JSON dengan status 400 dan pesan bahwa format JSON tidak valid.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /museum-collections/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	☒ Ya ☐ Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Request GET dengan ID 99 diteruskan ke Controller untuk mencari data pada Model. Karena koleksi dengan ID tersebut tidak ditemukan, Controller menghasilkan error 404 dan errorHandler mengirimkan response JSON dengan status 404.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Berhasil
T02	Menampilkan satu data berdasarkan ID	200	200	Berhasil
T03	Menambah data baru	201	201	Berhasil
T04	Mengubah data	200	200	Berhasil
T05	Menghapus data	204	204	Berhasil
N01	Menambah data tanpa API key	401	401	Berhasil ditolak
N02	Menambah data dengan body tidak lengkap	400	400	Berhasil ditolak
N03	Mengirim JSON yang rusak	400	400	Berhasil ditolak
N04	Mengambil ID yang tidak ada	404	404	Berhasil ditolak

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/VarrelFernando/si5b_kuis1_varrelfernando
Pesan commit terakhir	first commit
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route digunakan untuk menentukan alamat endpoint dan menghubungkannya dengan Controller, contohnya `router.get("/", museumController.getAllCollections);`. Controller menangani request, validasi, dan response, contohnya `const id = Number(req.params.id);`, sedangkan Model mengolah data koleksi museum, contohnya `return collections.find((item) => item.id === id);`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Error handler terpusat membuat penanganan error lebih rapi karena tidak perlu menulis response error yang sama di setiap route. Buktinya saat mengakses `/museum-collections/99`, server memberikan status 404 dengan pesan "Koleksi tidak ditemukan" melalui errorHandler, sehingga error dapat ditangani dari satu tempat.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	☒
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	☒
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	☒
4	Identitas saya (nama atau NIM) muncul pada data uji POST	☒
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	☒
6	Tabel rekapitulasi sesuai dengan tangkapan layar	☒
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	☒
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	☒

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100